

8.(라즈베리파이)

카메라 활용

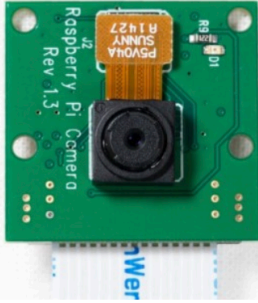
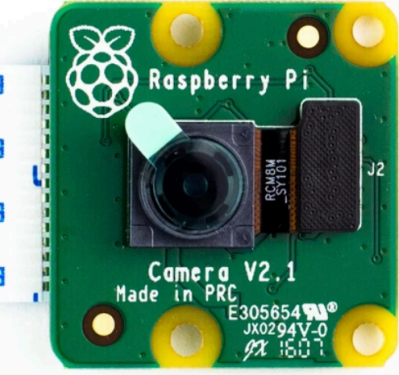
IoT 사물인터넷

학습목표

- 라즈베리 파이 카메라를 세팅할 수 있다.
- 라즈베리 파이 카메라를 활용하여 사진찍기, 동영상 촬영을 할 수 있다.
- OpenCV를 이용하여 모션감지 CCTV를 구현한다.

1. RPi 카메라

2.라즈베리파이 카메라 사양

Model	Raspberry pi camera V1	Raspberry pi camera V2	Raspberry pi camera HQ
Photo			
Net price	\$25	\$25	\$50
Size	Around 25 × 24 × 9 mm	Around 25 × 24 × 9 mm	38 x 38 x 18.4mm (excluding lens)
Weight	3g	3g	
Still resolution	5 Megapixels	8 Megapixels	12.3 Megapixels
Video modes	1080p30, 720p60 and 640 × 480p60/90	1080p30, 720p60 and 640 × 480p60/90	1080p30, 720p60 and 640 × 480p60/90
Linux integration	V4L2 driver available	V4L2 driver available	V4L2 driver available

1. RPi 카메라

1. 라즈베리 파이 카메라 세팅

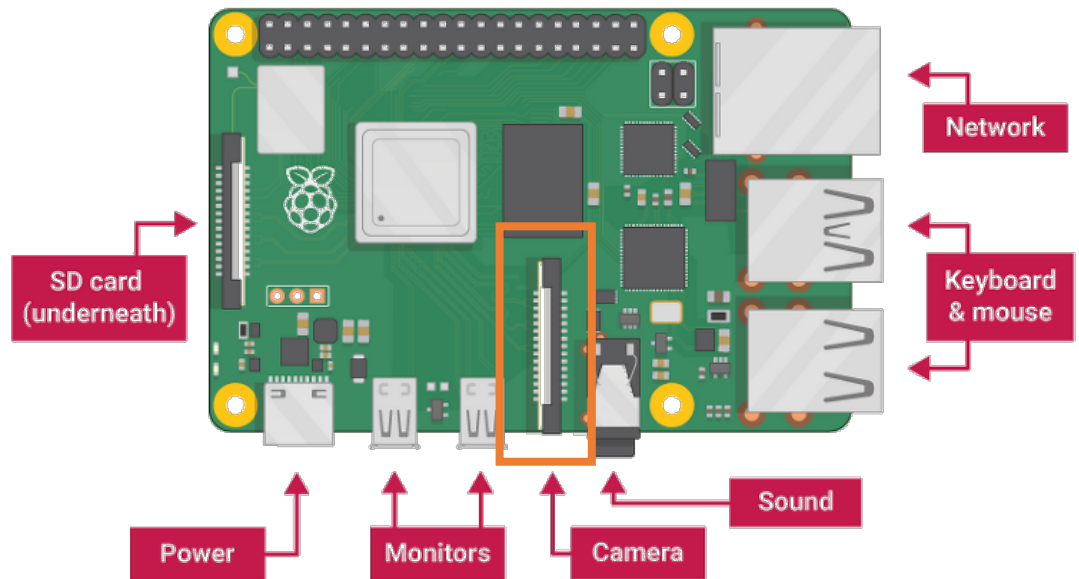
1) 라즈베리 파이 카메라 모듈은 라즈베리 파이의 공식 제품

: 카메라 모듈 v1부터 1080p 해상도를 지원

2) RPi 카메라 연결하기

: **(중요)** 연결 전 라즈베리파이 전원을 OFF

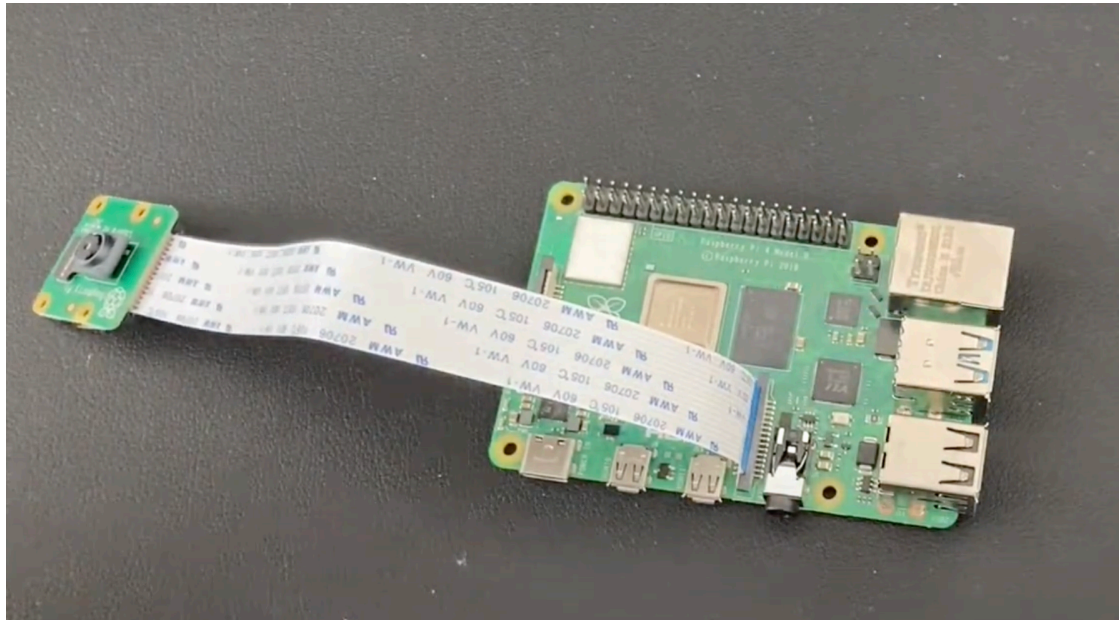
: 카메라 단자를 열고 케이블 연결
(단자의 핀 방향 확인)



1. RPi 카메라

1. 라즈베리파이 카메라 세팅

1) 카메라 연결 (전원 off 상태)

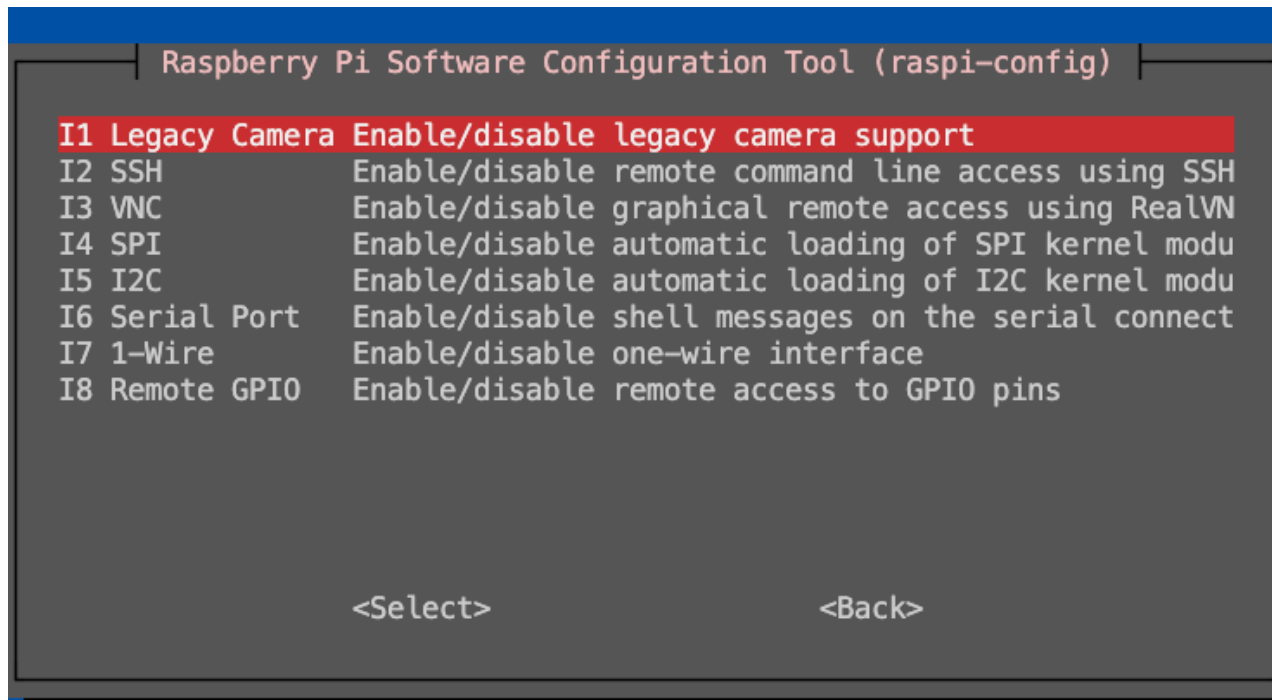


<https://youtu.be/yhM1NhD-kGs?t=26>

1. RPi 카메라

1. 라즈베리파이 카메라 세팅

2) 카메라 설정하기



: \$ sudo raspi-config

Interface Options - Legacy Camera - Yes - 확인 - Finish

\$ sudo reboot

1. RPi 카메라

2. 라즈베리 파이 카메라 테스트

```
$ vcgencmd get_camera  
supported=1 detected=1
```

1) 파이 카메라 포트

- : 프리뷰 포트 - 미리보기를 가능하게 해줌
- : 스틸 포트 - 사진을 찍을 수 있게 해줌
- : 비디오 포트 - 영상을 촬영할 수 있게 해줌

2) 사진 촬영하기 (home - pi - webapps - ch07)

- : \$ sudo raspistill -o test.jpg
- 5초 뒤 사진이 찍힘 (해당 폴더에 가서 확인)

3) 영상 촬영하기 (home - pi - webapps - ch07)

- : \$ sudo raspivid -o video.h264
- 5초 동안 촬영 (기본값)
- : 10초 동안 촬영
- \$ sudo raspivid -o video2.h264 -t 10000

1. RPi 카메라

3. 라즈베리 파이 카메라 명령어

1) help 명령어

: \$ raspistill --help

: \$ raspivie --help

2) raspistill 옵션

: -o 파일명 - 파일명으로 저장

: -t 시간 - 시간 후에 촬영 (단위 ms, 기본 5초)

: -w, -h - 사진 크기 (w 넓이, h 높이)

: -br - 밝기 조정 (0~100)

3) raspivid 옵션

: -o 파일명 - 파일명으로 저장

: -t 시간 - 시간 동안 촬영 (단위 ms, 기본 5초)

: -w, -h - 영상 크기 (w 넓이, h 높이)

: -br - 밝기 조정 (0~100)

- 기타 다양한 옵션들 (<https://wikidocs.net/3251> 구글 공식문서 번역)

2. 파이썬을 활용한 RPi 카메라

1. python-picamera 모듈 활용하기

1) 라이브러리 설치

: \$ sudo apt-get install python-picamera

(참고 - <https://projects.raspberrypi.org/en/projects/getting-started-with-picamera/4>)

2) 파이썬 코드 작성하기 (이미지)

```
1 import picamera
2 import time
3
4 # with...as 구문 : 파일스트림을 다루는데 자동으로 open, close
5 with picamera.PiCamera() as camera:
6     camera.rotation = 180          #화면을 180도 회전
7     camera.resolution = (640, 480) #화면사이즈
8     camera.start_preview()         #미리보기(네트워크접속하면 보이지 않음)
9     camera.image_effect = 'sketch' #기본적으로 제공하는 이미지 이펙트
10    time.sleep(5)
11    camera.capture('pi_still.jpg')
12    camera.stop_preview()
```

2. 파이썬을 활용한 RPi 카메라

1. python-picamera 모듈 활용하기

3) 파이썬 코드 작성하기 (영상)

```
1 import picamera
2
3 # with...as 구문 : 파일스트림을 다루는데 자동으로 open, close
4 with picamera.PiCamera() as camera:
5     camera.rotation = 180          #화면을 180도 회전
6     camera.resolution = (640, 480) #화면사이즈
7     #미리보기(네트워크접속하면 보이지 않음)
8     camera.start_preview()
9     camera.start_recording('pi_mov.h264')
10    camera.wait_recording(30)
11    camera.stop_recording()
12    camera.stop_preview()
```

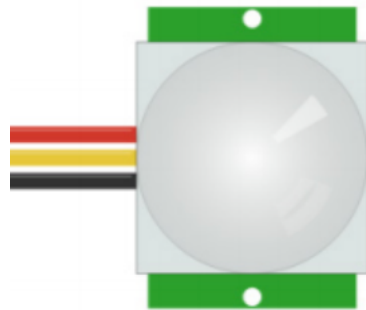
2. 파이썬을 활용한 RPi 카메라

4) 실습 - 버튼을 누르면 사진 촬영하기

5) 실습2 - 상태 LED 적용

: 평소 빨간 LED가 들어오다가 촬영버튼을 누르면 3초간 노란 LED가 들어오다 다시 빨간 LED 점등

6) 실습3 - 인체감지 센서가 작동하면 그 당시 시간으로 이미지 저장하기



적외선 감지센서(HC-SR501)

3. OpenCV를 활용한 색상검출



1. OpenCV 란?

1) 실시간 컴퓨터 비전을 목적으로 한 프로그래밍 라이브러리.

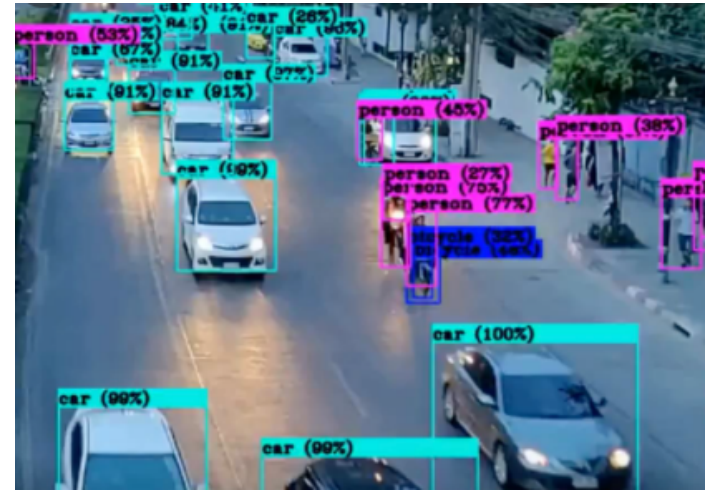
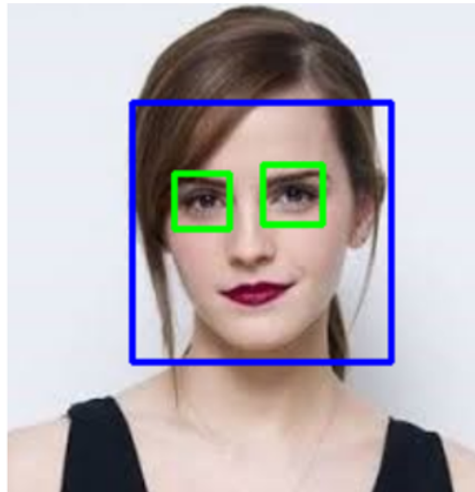
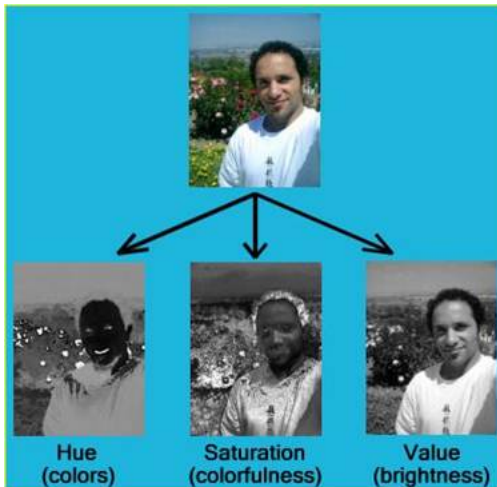
인텔이 개발하였으며, 실시간 이미지 프로세싱에 중점을 둔 라이브러리

: 공장에서 제품을 검사할 때

: 의료 영상 처리 및 보정, 판단

: CCTV영상

: 머신러닝



3. OpenCV를 활용한 색상검출

1. OpenCV 란?

2) python3용 OpenCV 및 관련 라이브러리 설치

```
$ sudo pip3 install opencv-python==4.5.1.48
```

```
$ sudo pip3 install numpy==1.20.2    #수치계산할때 사용할 라이브러리
```

```
$ sudo apt-get install libhdf5-dev -y
```

```
$ sudo apt-get install libhdf5-serial-dev -y
```

```
$ sudo apt-get install libatlas-base-dev -y
```

```
$ sudo apt-get install libjasper-dev -y
```

```
$ sudo apt-get install libqtgui4 -y
```

```
$ sudo apt-get install libqt4-test -y
```

3. OpenCV를 활용한 색상검출

1. OpenCV 활용하기

1) OpenCV를 활용한 영상 출력

```
1 import cv2
2
3 def main():
4     #카메라를 비디오로 불러와 camera 객체와 바인딩한다.
5     camera = cv2.VideoCapture(0)
6     #영상의 크기를 결정
7     #3은 cv2.CAP_PROP_FRAME_WIDTH / 4는 cv2.CAP_PROP_FRAME_HEIGHT
8     camera.set(3,320)
9     camera.set(4,240)
10
11     while(1):
12         _, frame = camera.read()
13         #이미지를 출력
14         cv2.imshow('Frame', frame)
15         #키보드 q키를 누르면 종료
16         if cv2.waitKey(1) == ord('q'):
17             break
18
19     camera.release()
20     cv2.destroyAllWindows()
21
22 if __name__ == '__main__':
23     main()
```

3. OpenCV를 활용한 색상검출

1. OpenCV 활용하기

1) 색상 검출 (HSV / 색상, 채도, 명도 활용)

```
1  import cv2
2  import numpy as np
3
4  def main():
5      camera = cv2.VideoCapture(0)
6      camera.set(3,320)
7      camera.set(4,240)
8
9      while(1):
10         _, frame = camera.read()
11
12         #색상 공간 변환 함수 cvtColor() RGB to HSV
13         hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
14
15         #컬러값의 범위를 설정(H색상, S채도, V명도)
16         lower_blue = (100,100,120)
17         upper_blue = (150, 255, 255)
18
19         lower_green = (50, 150, 50)
20         upper_green = (80, 255, 255)
21
22         lower_red = (150, 50, 50)
23         upper_red = (180, 255, 255)
```

3. OpenCV를 활용한 색상검출

1. OpenCV 활용하기

1) 색상 검출 (HSV / 색상, 채도, 명도 활용)

```
25     #해당 영역의 색상값만 추출 (추출된 영역이 화이트로 보임)
26     redMask = cv2.inRange(hsv, lower_red, upper_red)
27     greenMask = cv2.inRange(hsv, lower_green, upper_green)
28     blueMask = cv2.inRange(hsv, lower_blue, upper_blue)
29
30     #이미지 비트 연산(mask영역에서 서로 공통으로 겹치는 부분 출력)
31     red = cv2.bitwise_and(frame, frame, mask=redMask)
32     green = cv2.bitwise_and(frame, frame, mask=greenMask)
33     blue = cv2.bitwise_and(frame, frame, mask=blueMask)
34
35     #이미지를 출력
36     cv2.imshow('Frame', frame)
37     cv2.imshow('Red', redMask)
38     cv2.imshow('Green', greenMask)
39     cv2.imshow('Blue', blueMask)
40
41     #키보드 q키를 누르면 종료
42     if cv2.waitKey(1) == ord('q'):
43         break
44
45     camera.release()
46     cv2.destroyAllWindows()
```

3. OpenCV를 활용한 색상검출

2) 빨간색, 녹색, 파란색의 값 수치화

```
blueMask = cv2.inRange(hsv, lower_blue, upper_blue)

redPixels = cv2.countNonZero(redMask)
greenPixels = cv2.countNonZero(greenMask)
bluePixels = cv2.countNonZero(blueMask)

colorList = [redPixels, greenPixels, bluePixels]
maxValue = max(colorList)
maxPos = colorList.index(maxValue)

print(maxValue, maxPos)
print(redPixels, greenPixels, bluePixels)
```

3. OpenCV를 활용한 색상검출

[실습] Red, Green, Blue LED 회로를 구성하고,

maxValue >= **500** 이상일때

해당 LED On 나머지 LED off

maxValue 500 미만일 경우는

전체 LED off

maxValue **300**
 1000

3. OpenCV를 활용한 얼굴인식

```
cd /home/pi
```

```
git clone https://github.com/opencv/opencv.git
```

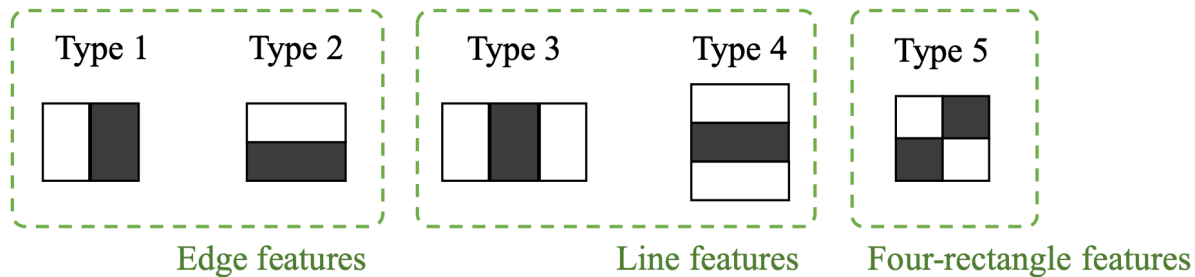
얼굴인식을 사용하기 위해서 OpenCV에서는 알고리즘
이 적용된 haarcascades 파일을 xml 형태로 제공
(하르 캐스케이드)

```
git clone 후  
/home/pi/opencv/data/haarcascades 폴더 확인
```

캐스케이드 분류기 : 얼굴 검출

유사 하르 특징(Haar-like features)

- 사각형 형태의 필터 집합을 사용
- 흰색 사각형 영역 픽셀 값의 합에서 검정색 사각형 영역 픽셀 값을 뺀 결과 값을 추출



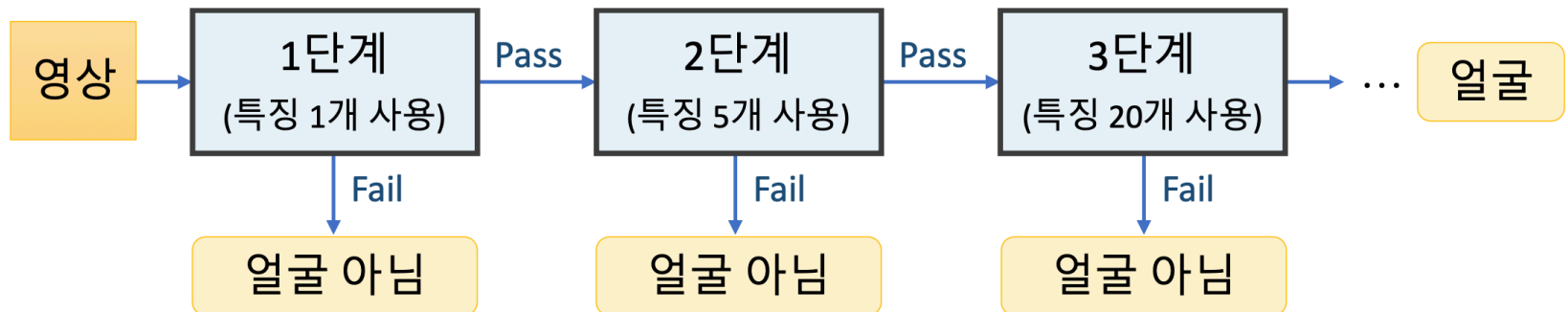
- 24 x 24 부분 영상에서 얼굴 판별에 유용한 유사 하르 특징을 선별



캐스케이드 분류기 : 얼굴 검출

캐스케이드 분류기(Cascade classifier)

- 일반적인 영상에는 얼굴이 한 두개 있을 뿐, 나머지 영역은 대부분 non-face 영역
- Non-face 영역을 빠르게 skip하도록 다단계 검사 수행



캐스케이드 분류기 : 얼굴 검출

캐스케이드 분류기 얼굴 검출 과정 시각화



<https://www.youtube.com/watch?v=hPCTwxFOqf4>

캐스케이드 분류기 : 얼굴 검출

■ cv2.CascadeClassifier 객체 생성 및 학습 데이터 불러오기

```
cv2.CascadeClassifier(      ) -> <CascadeClassifier object>  
cv2.CascadeClassifier(filename) -> <CascadeClassifier object>
```

```
cv2.CascadeClassifier.load(filename) -> retval
```

- filename: XML 파일 이름
- retval: 성공하면 True, 실패하면 False.

캐스케이드 분류기 : 얼굴 검출

미리 학습된 xml 파일 다운로드

<https://github.com/opencv/opencv/tree/master/data/haarcascades>

XML 파일 이름	검출 대상
haarcascade_frontalface_default.xml haarcascade_frontalface_alt.xml haarcascade_frontalface_alt2.xml haarcascade_frontalface_alt_tree.xml	정면 얼굴 검출
haarcascade_profileface.xml	측면 얼굴 검출
haarcascade_smile.xml	웃음 검출
haarcascade_eye.xml haarcascade_eye_tree_eyeglasses.xml haarcascade_lefteye_2splits.xml haarcascade_righteye_2splits.xml	눈 검출
haarcascade_frontalcatface.xml haarcascade_frontalcatface_extended.xml	고양이 얼굴 검출
haarcascade_fullbody.xml	사람의 전신 검출
haarcascade_upperbody.xml	사람의 상반신 검출
haarcascade_lowerbody.xml	사람의 하반신 검출
haarcascade_russian_plate_number.xml haarcascade_licence_plate_rus_16stages.xml	러시아 자동차 번호판 검출

캐스케이드 분류기 : 얼굴 검출

■ CascadeClassifier 멀티스케일 객체 검출 함수

```
cv2.CascadeClassifier.detectMultiScale(image, scaleFactor=None,  
    minNeighbors=None, flags=None, minSize=None, maxSize=None) -> result
```

- image: 입력 영상 (cv2.CV_8U)
- scaleFactor: 영상 축소 비율. 기본값은 1.1.
- minNeighbors: 얼마나 많은 이웃 사각형이 검출되어야 최종 검출 영역으로 설정할지를 지정. 기본값은 3.
- flags: (현재) 사용되지 않음
- minSize: 최소 객체 크기. (w, h) 튜플.
- maxSize: 최대 객체 크기. (w, h) 튜플.
- result: 검출된 객체의 사각형 정보(x, y, w, h)를 담은 `numpy.ndarray`.
shape=(N, 4). dtype=`numpy.int32`.

3. OpenCV를 활용한 얼굴인식

1. OpenCV 활용하기 (Cascading classifiers)

```
import cv2
import time

def main():
    camera = cv2.VideoCapture(0)
    camera.set(3, 640)
    camera.set(4, 480)

    xml = "/home/pi/opencv/data/haarcascades/haarcascade_frontalface_default.xml"
    face_cascade = cv2.CascadeClassifier(xml)

    while camera.isOpened():
        _, image = camera.read()
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

        faces = face_cascade.detectMultiScale(gray, 1.05, 5)
        print("faces detected Number: " + str(len(faces)))
```

3. OpenCV를 활용한 얼굴인식

1. OpenCV 활용하기 (Cascading classifiers)

```
if len(faces):
    for x, y, w, h in faces:
        cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

    cv2.imshow("result", image)

    time.sleep(0.2)
    if cv2.waitKey(1) == ord("q"):
        break

cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

3. OpenCV를 활용한 얼굴인식

1. OpenCV 활용하기

[문제] 얼굴 사각형을 blur 처리

```
for x, y, w, h in faces:  
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)  
  
    # blur
```

3. OpenCV를 활용한 얼굴인식

2. OpenCV 활용하기 - 줄음방지 알리미 만들기

```
import cv2

def main():
    camera = cv2.VideoCapture(-1)
    camera.set(3,640)
    camera.set(4,480)

    face_xml = '/home/pi/opencv/data/haarcascades/haarcascade_frontalface_default.xml'
    eye_xml = '/home/pi/opencv/data/haarcascades/haarcascade_eye.xml'

    face_cascade = cv2.CascadeClassifier(face_xml)
    eye_cascade = cv2.CascadeClassifier(eye_xml)

    while( camera.isOpened() ):
        _, image = camera.read()
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

        faces = face_cascade.detectMultiScale(gray,scaleFactor=1.1,
                                                minNeighbors=5,minSize=(100,100),flags=cv2.CASCADE_SCALE_IMAGE)
        print("faces detected Number: " + str(len(faces)))
```

3. OpenCV를 활용한 얼굴인식

2. OpenCV 활용하기 - 줄음방지 알리미 만들기

```
if len(faces):
    for (x,y,w,h) in faces:
        cv2.rectangle(image,(x,y),(x+w,y+h),(255,0,0),2)

        face_gray = gray[y:y+h, x:x+w]
        face_color = image[y:y+h, x:x+w]

        eyes = eye_cascade.detectMultiScale(face_gray,scaleFactor=1.1,
                                              minNeighbors=5)

        for (ex,ey,ew,eh) in eyes:
            cv2.rectangle(face_color, (ex, ey), (ex+ew, ey+eh), (0,255,0), 2)

    cv2.imshow('result', image)

    if cv2.waitKey(1) == ord('q'):
        break

cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

3. OpenCV를 활용한 얼굴인식

2. OpenCV 활용하기 - 졸음방지 알리미 만들기

[문제] 졸음 감지시 부저 울리기



얼굴을 찾았는데 눈을 찾지 못하면 알람을 울린다.

3. OpenCV를 활용한 모션 감지 CCTV

3. 모션감지 CCTV

[따라하기] 카메라 영상의 차영상으로 움직임을 감지하고 움직임을 있는 영역을 표시하는 프로그램을 만들어 보세요.

[힌트] 이전과 이후의 두 프레임 영상에서 단순히 차이를 구해서는 움직임을 감지할 수 없습니다.

카메라를 고정한 책상이나 건물이 미세하게 떨리기도 하고 카메라의 이미지 센서(CCD)의 한계 때문에 영상 간의 차이는 반드시 항상 발생합니다. 따라서 세 프레임 a, b, c 를 순차적으로 얻어서 a와 b의 차이가 모두 발견되는 경우에 한해서 움직임을 있는 것으로 판단해야 합니다.

3. OpenCV를 활용한 모션 감지 CCTV

3. 모션감지 CCTV

이때 각 프레임의 차이가 특정 기준치보다 작은 픽셀은 무시하고 차이가 없는 것으로 간주합니다.

이렇게 해서 최종적으로 차이가 있는 것으로 판단한 픽셀의 개수가 특정 기준치보다 많은 경우에 움직임이 있는 것으로 간주합니다.

3. OpenCV를 활용한 모션 감지 CCTV

```
import cv2
import numpy as np

# 감도 설정 (카라 품질에 따라 기준치 설정)
thresh = 25
max_diff = 5

a, b, c = None, None, None
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 480)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 320)

if cap.isOpened():
    ret, a = cap.read()
    ret, b = cap.read()

    while ret:
        ret, c = cap.read()
        draw = c.copy()
        if not ret:
            break
```

3. OpenCV를 활용한 모션 감지 CCTV

```
# 3개의 영상을 그레이 스케일로 변경
a_gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
b_gray = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)
c_gray = cv2.cvtColor(c, cv2.COLOR_BGR2GRAY)

diff1 = cv2.absdiff(a_gray, b_gray)
diff2 = cv2.absdiff(b_gray, c_gray)

# 스레드홀드로 기준치 이내의 차이는 무시
ret, diff1_t = cv2.threshold(diff1, thresh, 255, cv2.THRESH_BINARY)
ret, diff2_t = cv2.threshold(diff2, thresh, 255, cv2.THRESH_BINARY)

# 두 차이에 대해서 AND연산, 두 영상의 차이가 모두 발견된 경우
diff = cv2.bitwise_and(diff1_t, diff2_t)

# 열림 연산으로 노이즈 제거
k = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))
diff = cv2.morphologyEx(diff, cv2.MORPH_OPEN, k)

diff_cnt = cv2.countNonZero(diff)
```

3. OpenCV를 활용한 모션 감지 CCTV

```
# 차이가 발생한 픽셀이 개수 판단 후 사각형 그리기
if diff_cnt > max_diff:
    if diff_cnt > max_diff:
        nzero = np.nonzero(diff)
        cv2.rectangle(
            draw,
            (min(nzero[1]), min(nzero[0])),
            (max(nzero[1]), max(nzero[0])),
            (0, 255, 0),
            2,
        )
        cv2.putText(
            draw,
            "Motion Detected",
            (10, 30),
            cv2.FONT_HERSHEY_DUPLEX,
            0.5,
            (0, 0, 255),
        )
```

3. OpenCV를 활용한 모션 감지 CCTV

```
# 컬러 스케일 영상과 스레시홀드 영상을 통합해서 출력
stacked = np.hstack((draw, cv2.cvtColor(diff, cv2.COLOR_GRAY2BGR)))
cv2.imshow("motion sensor", stacked)

a = b
b = c

if cv2.waitKey(1) & 0xFF == 27:
    break
```

3. OpenCV를 활용한 모션 감지 CCTV

[문제] 모션 감지시 파일 저장

image 폴더 만들기

(탐색기에서 소스 디렉토리에 image 디렉토리생성)

./image/YYYY-mm-dd_HHMMSS.jpg 이미지 이름 포맷

3. OpenCV를 활용한 얼굴인식

4. OpenCV 활용하기 - Live Video Streaming

```
import cv2
from flask import Flask, render_template, Response

app = Flask(__name__)
# 가중치 파일 경로
cascade_filename = "/home/pi/opencv/data/haarcascades/haarcascade_frontalface_alt.xml"
# 모델 불러오기
cascade = cv2.CascadeClassifier(cascade_filename)
```

3. OpenCV를 활용한 얼굴인식

4. OpenCV 활용하기 - Live Video Streaming

```
def gen():
    camera = cv2.VideoCapture(0)
    camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    while 1:
        _, frame = camera.read()
        retImg = imgDetector(frame, cascade)
        cv2.imwrite("pic.jpg", retImg)

        yield (
            b"--frame\r\n"
            b"Content-Type: image/jpeg\r\n\r\n" + open("pic.jpg", "rb").read() + b"\r\n"
        )

    camera.release()
    cv2.destroyAllWindows()
```

3. OpenCV를 활용한 얼굴인식

4. OpenCV 활용하기 - Live Video Streaming

```
@app.route("/")
def index():
    """Video streaming ."""
    return render_template("index.html")

@app.route("/video_feed")
def video_feed():
    """Video streaming route. Put this in the src attribute of an img tag."""
    return Response(gen(), mimetype="multipart/x-mixed-replace; boundary=frame")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=9000, debug=True, threaded=True)
```

3. OpenCV를 활용한 얼굴인식

4. OpenCV 활용하기 - Live Video Streaming

```
def imgDetector(img, cascade):  
    # 영상 압축  
    img = cv2.resize(img, dsize=None, fx=1.0, fy=1.0)  
    # 그레이 스케일 변환  
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
    # cascade 얼굴 탐지 알고리즘  
    results = cascade.detectMultiScale(  
        gray, # 입력 이미지  
        scaleFactor=1.5, # 이미지 피라미드 스케일 factor  
        minNeighbors=5, # 인접 객체 최소 거리 픽셀  
        minSize=(20, 20), # 탐지 객체 최소 크기  
    )  
  
    for box in results:  
        x, y, w, h = box  
        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 255, 255), thickness=2)  
  
    return img
```

3. OpenCV를 활용한 얼굴인식

5. OpenCV 활용하기 - 자유 프로젝트 (3시간) -

